

NUMÉRIQUE et SCIENCES INFORMATIQUES

Partie pratique: Sujet n°14

Question 1 :

```
def nb_occupants_restants(self) -> int:
    """ renvoie le nombre d'occupants restants dans la pièce.
        A FAIRE EN PARTIE 1
    """
    res=0
    for i in self.grille:
        for j in i:
            res += j
    return res
```

→ On crée un compteur res qu'on initialise à 0, puis parcourt chaque case de la grille, ligne par ligne, case par case, et au fur et à mesure on rajoute la valeur de la case au résultat qui représente le nombre d'occupant de la grille. On renvoie à la fin de la fonction, res ,correspondant donc au nombre total d'occupant.

Question 2 :

```
def evacuation(p: Piece, silencieux: bool = True) -> int:
    """
    simule l'évacuation de la pièce et renvoie le nombre de tours nécessaire.
    A chaque tour, chacun des occupants se déplace, si possible, d'une case
    vers la sortie la plus proche. Si le paramètre silencieux vaut false,
    l'état de la pièce à chaque tour est affiché dans la console.
    A FAIRE EN PARTIE 1
    """
    tour=0
    while p.nb_occupants_restants()>0:
        p.alerter()
        tour+=1
    return tour
```

→ On crée un compteur tour qu'on initialise à 0, tant qu'il reste des occupants (nombre d' occupants > 0), on fait appelle à la fonction alerte qui fait déplacer les occupants vers une sortie et on compte le nombre déplacement en ajoutant à tour +1 . On renvoie à la fin de la fonction le nombre de tour total qui a permis aux occupants de tous évacuer.

Question 3 :

```
def ajouter_sortie(self, direction: str, position: int):
    """ permet d'ajouter des sorties à la pièce.
        A COMPLETER EN PARTIE 2 (Pour l'instant, on n'utilise que deux directions !)
    """
    if direction == "N":
        self.sorties.append((0, position))
    elif direction == "O":
        self.sorties.append((position, 0))
```

```

elif direction == "S":
    self.sorties.append((self.i_max, position))
elif direction == "E":
    self.sorties.append((position,self.j_max))

```

→ La direction sud correspond à la dernière ligne de la grille (self.i_max) sur n'importe quelle colonne.
 La direction est correspond à la dernière colonne de la grille (self.j_max) sur n'importe quelle ligne.

Question 4 :

```

def choix_sortie(self, i, j) -> tuple:
    """ renvoie la sortie à utiliser pour une personne positionnée sur la ligne i et la colonne j.
        A CORRIGER EN PARTIE 2 (Pour l'instant, seule la 1ère sortie est utilisée !)
    """
    def distance_de_manhattan(destination):
        """ fonction privée renvoyant la distance de Manhattan entre la case (i,j) et la destination reçue en
            paramètre.
        """
        return abs(i - destination[0]) + abs(j - destination[1])
    assert len(self.sorties) > 0, "Aucune sortie"
    choix = self.sorties[0]
    distance = distance_de_manhattan(choix)
    for k in range(1, len(self.sorties)):
        autre_sortie = self.sorties[k]
        d2 = distance_de_manhattan(autre_sortie)
        if d2 < distance:
            choix = autre_sortie
            distance = d2
    return choix

```

→ k commence obligatoirement par 1 et donc ne peut pas être inférieur à 0, d2 n'était pas prédéfinie.